Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Разработка модулей ядра операционной системы для динамического перехвата системных вызовов с использованием технологии eBPF»

Выполнил:

Студент 4 курса

Группы ИИ-26

Пасевич К.Ю.

Проверила:

Хацкевич А.С.

Брест 2026

Цель работы: практическое освоение методов разработки программ расширенного фильтра пакетов Беркли (eBPF) на языке C для мониторинга событий уровня ядра, динамического перехвата системных вызовов (kprobes/tracepoints) и выявления аномального поведения процессов в операционной системе Linux с использованием фреймворка BCC (BPF Compiler Collection).

Ход работы

Задание 1. Базовый перехват запуска процессов

1. Разработайте eBPF-зонд (C + Python), использующий механизм перехвата kprobe (или tracepoint) для отслеживания системного вызова execve (запуск новых процессов).
2. Напишите логику на стороне ядра (C), которая извлекает идентификатор процесса (PID), идентификатор пользователя (UID) и имя исполняемого файла (comm).
3. Запустите скрипт с правами root и продемонстрируйте перехват запуска стандартных утилит (например, ls или cat) в соседнем окне терминала или новой вкладке WSL.

Задание 2. Динамический анализ аномального поведения (Индивидуальный вариант)

1. Определите целевой системный вызов и условие аномального поведения согласно вашему индивидуальному варианту.
2. Напишите eBPF-зонд на C, извлекающий аргументы выбранного системного вызова (например, путь к целевому файлу, целевой PID или сетевой порт).
3. Реализуйте логику фильтрации строго на стороне ядра: программа должна передавать событие в пространство пользователя (через кольцевой буфер BPF_PERF_OUTPUT) *только* при совпадении с условием аномалии. Передача всех подряд событий в Python-скрипт является грубой ошибкой производительности.
4. Сгенерируйте целевую аномалию (например, попытайтесь прочитать защищенный файл) и зафиксируйте успешный перехват алерта вашим скриптом.

Задание 3. Интеграция с пространством пользователя и аудит

1. Расширьте Python-скрипт из задания 2: при получении события об аномалии из ядра скрипт должен автоматически записывать предупреждение в локальный лог-файл security_audit.log.
2. Формат записи должен содержать: временную метку (Timestamp), PID нарушителя, UID нарушителя и детали события.

Задание 4. Проектирование профилей защиты (Без написания кода)

1. Сравните механизмы изоляции Seccomp-BPF и eBPF (kprobes) применительно к задаче блокировки системных вызовов. Обоснуйте, почему стандартный eBPF-зонд преимущественно используется для мониторинга и аудита, а не для предотвращения (блокировки) действий вредоносного ПО.

2. Опишите концепцию атаки состояния гонки типа TOCTOU (Time-of-Check to Time-of-Use) применительно к вашему eBPF-зонду из задания 2 и предложите архитектурный метод противодействия.

Задание 1. Базовый перехват запуска процессов

Что делали. Разработали eBPF-зонд из двух частей: C-код, загружаемый в ядро, и Python-обёртка для чтения событий. C-часть привязывается через kprobe к системному вызову execve, извлекает PID процесса через bpf_get_current_pid_tgid(), UID через bpf_get_current_uid_gid() и имя процесса через bpf_get_current_comm(). Данные упаковываются в структуру и отправляются в user space через кольцевой буфер BPF_PERF_OUTPUT вызовом perf_submit. Python-часть подписывает функцию-обработчик на кольцевой буфер и в цикле читает события через perf_buffer_poll().

Что вводили. В первом окне WSL запускали монитор командой sudo python3 monitor_template.py. Во втором окне выполняли ls, cat /etc/hostname, whoami для генерации вызовов execve.

Что получили. Монитор вывел строки вида:

```
10 warnings generated.
[*] Мониторинг запущен. Ожидание вызовов execve... (Нажмите Ctrl+C для выхода)
Перехват! PID: 679, UID: 1000, COMM: Relay
Перехват! PID: 680, UID: 1000, COMM: bash
Перехват! PID: 681, UID: 1000, COMM: bash
Перехват! PID: 684, UID: 1000, COMM: bash
Перехват! PID: 685, UID: 1000, COMM: bash
Перехват! PID: 686, UID: 1000, COMM: bash
Перехват! PID: 687, UID: 1000, COMM: bash
Перехват! PID: 688, UID: 1000, COMM: lesspipe
Перехват! PID: 690, UID: 1000, COMM: lesspipe
Перехват! PID: 691, UID: 1000, COMM: bash
Перехват! PID: 692, UID: 1000, COMM: bash
Перехват! PID: 693, UID: 1000, COMM: bash
Перехват! PID: 694, UID: 1000, COMM: bash
```

Задание 2. Динамический анализ аномального поведения

Условие варианта 10. Целевой системный вызов: sys_enter_init_module. Условие аномалии: любая попытка загрузки модуля ядра (.ko) в систему.

Что делали. Разработали второй eBPF-зонд, привязанный к загрузке модулей ядра. Первоначально планировалось использовать kprobe на sys_enter_init_module, но в WSL 2 функция __x64_sys_enter_init_module отсутствует в таблице символов ядра, и привязка падала с ошибкой. Решение: использовать статическую точку трассировки (tracepoint) sys_enter_finit_module, доступную в WSL. Современные утилиты insmod и modprobe используют именно finit_module для загрузки модуля по файловому дескриптору, так что tracepoint полностью покрывает задачу варианта. Логика фильтрации в ядре: поскольку условие варианта это любая попытка загрузки модуля, событие отправляется при каждом срабатывании tracepoint без дополнительных проверок.

Что вводили. Запускали монитор в первом окне командой sudo python3 monitor_init_module.py. Во втором окне сначала выгружали модуль dummy (sudo rmmod dummy), затем загружали его заново (sudo modprobe dummy). Это гарантированно вызывает finit_module.

Что получили. В первом окне появилась строка:

```
10 warnings generated.
[*] Мониторинг init_module запущен. Ctrl+C для выхода.
[ALERT] PID: 1201, UID: 0, COMM: modprobe
```

Задание 3. Интеграция с пространством пользователя и аудит

Что делали. Расширили Python-скрипт из Задания 2: добавили импорт datetime и логику записи алертов в файл security_audit.log. Таймстамп формируется через datetime.datetime.now().strftime() в формате YYYY-MM-DD HH:MM:SS. Файл открывается в режиме "a" (append), чтобы каждая новая аномалия добавляла строку, не перезаписывая предыдущие. Формат записи: таймстамп, PID, UID, имя процесса и метка события.

Что вводили. После запуска монитора и генерации аномалии через sudo modprobe dummy останавливали монитор (Ctrl+C) и проверяли содержимое лог-файла командой cat ~/projects/lab3/security_audit.log.

Что получили. Файл содержит запись:

```
10 warnings generated.
[*] Мониторинг init_module запущен. Ctrl+C для выхода.
[ALERT] 2026-10-13 21:45:05 | PID: 1283 | UID: 0 | COMM: modprobe
```

Задание 4. Проектирование профилей защиты

4.1. Сравнение Seccomp-BPF и eBPF kprobes

Seccomp-BPF работает как фильтр на входе в системный вызов. Процесс объявляет белый или чёрный список разрешённых syscall'ов ещё до их исполнения. Ядро проверяет вызов до того, как он начнёт выполняться, и может прервать его, вернув ошибку или завершив процесс. Это безопасный механизм блокировки.

eBPF kprobes прикрепляются к уже исполняющейся функции ядра. На момент срабатывания зонда ядро уже вошло в функцию, изменило глобальные структуры, взяло блокировки, выделило ресурсы. Программа eBPF может только наблюдать и читать данные через bpf_probe_read_*, но не может отменить вызов, потому что верификатор не позволит изменить логику ядра. Попытка прервать syscall на полпути оставит систему в несогласованном состоянии: утечки памяти, зависшие блокировки, «зависшие» структуры ядра. Поэтому eBPF применяется для мониторинга и аудита, а для блокировки используют Seccomp-BPF (на входе в syscall) или BPF-LSM hooks (в точке проверки прав доступа).

4.2. Уязвимость TOCTOU

TOCTOU (Time-of-Check to Time-of-Use) это класс уязвимостей, при которых между моментом проверки условия и моментом использования ресурса объект успевает измениться. В контексте eBPF-зонда: программа читает аргумент системного вызова, то есть указатель на user space память (например, путь к файлу или имя модуля). Через bpf_probe_read_user_str() eBPF копирует строку в буфер ядра, проверяет её и принимает решение. Атакующий в этот момент может подменить содержимое памяти по указателю: eBPF увидит одно (легитимное), а ядро обработает другое (вредоносное).

Метод противодействия. Всегда использовать bpf_probe_read_user_str() вместо прямого разыменования указателя: функция атомарно копирует данные из user space в буфер ядра, и eBPF работает со снимком, а не с живым указателем. Минимизировать окно между чтением и решением. Кэшировать прочитанные данные в BPF-карте при повторных обращениях. Для критичных решений использовать BPF-LSM hooks вместо kprobes, там TOCTOU-окно сведено к нулю архитектурно.

Вывод: получила практическое освоение методов разработки программ расширенного фильтра пакетов Беркли (eBPF) на языке C для мониторинга событий уровня ядра, динамического перехвата системных вызовов (kprobes/tracepoints) и выявления аномального поведения процессов в операционной системе Linux с использованием фреймворка BCC (BPF Compiler Collection).